

LightLLM 投机解码技术开发手册

大小模型投机解码的原理、发展历程、架构设计与工程实现

LightLLM 工程组

2026 年 8 月 11 日

目标 GPU: NVIDIA RTX 2060 (sm_75, 6GB 显存)

模型: Qwen2.5-0.5B / Qwen2.5-1.5B (fp16)

目录

1	引言	3
2	自回归解码的瓶颈与投机解码的动机	3
2.1	为什么自回归解码慢	3
2.2	降低成本的两种思路	3
3	投机解码原理	3
3.1	核心思想	3
3.2	草稿阶段：逐个自回归生成	4
3.3	验证阶段：一次批量前向	4
3.4	接受判据	4
3.4.1	严格 <code>argmax</code> 接受	4
3.4.2	概率接受 (Leviathan)	4
3.4.3	草稿必须采样而非贪心	5
3.5	残差修正与输出分布守恒定理	5
3.6	连续前缀约束	6
3.7	加速比的理论模型	6
4	投机解码的发展历程	6
4.1	早期并行解码尝试 (2018–2021)	6
4.2	投机解码的独立提出 (2022–2023)	6
4.3	工程化落地 (2023–2024)	7
4.4	后续演进 (2024 至今)	7
4.5	与本研究的关系	7
5	系统架构设计	7
5.1	分层结构	7
5.2	<code>DraftEngine</code> 抽象	8
5.3	批量验证原语 <code>forward_batched</code> 与 <code>run_layers</code>	8
5.4	KV cache 一致性	8
5.5	结构化输出 (Grammar) 集成	8
5.6	采样参数化	8
6	显存管理	9
7	实验与结果	9
7.1	自投机正确性 ($M = 24$)	9
7.2	0.5B 到 1.5B 双模型：接受判据的影响	9
7.3	测试矩阵	10
8	关键发现与设计决策	10
8.1	GEMM 批量尺寸敏感	10
8.2	<code>layer-slice</code> 自投机在深层小模型上的局限	10

8.3	草稿采样是接受率的分水岭	10
8.4	verify-as-decode: 消除双计算（已实现）	10
9	已知限制与未来工作	11
10	结论	11
A	配置项	11
B	测试清单	12

1 引言

大语言模型（LLM）的自回归生成是**逐 token 串行**的：每一步只产生一个 token，且每一步都必须把全部模型权重加载一遍才能计算下一个 token。在显存带宽受限的消费级 GPU 上，权重加载耗时远大于计算耗时，解码吞吐被**带宽锁死**，与模型规模近似成反比。这一本质瓶颈促使了投机解码（Speculative Decoding）的诞生：用一个小模型“猜”多个 token，再让大模型一次批量验证，从而以更少的权重加载次数产出更多 token，且**不改变模型的输出分布**。

LightLLM 在自研 C++/CUDA 推理引擎上完整实现了投机解码的工程化闭环，覆盖自投机（self-speculation）与双模型（dual-model）两条路径，并解决了 KV cache 一致性、显存预算、结构化输出等实际问题。本文档作为技术开发手册，从原理出发，梳理其发展历程，展开架构设计与关键实现，并以实测数据验证各项设计决策。

2 自回归解码的瓶颈与投机解码的动机

2.1 为什么自回归解码慢

自回归解码的核心循环为：输入前缀 $\rightarrow$ 前向传播 $\rightarrow$ 采样下一个 token $\rightarrow$ 追加到前缀 $\rightarrow$ 重复。每一步：

1. 权重矩阵总量为 W ，需从显存加载一次（或分片多次）；
2. 当前步只产生 1 个 token；
3. 每一步间存在串行依赖，无法并行。

在带宽受限设备上，单步耗时近似为 W/B （ B 为显存带宽）。因此吞吐 $\approx B/W$ ，即**权重越大越慢**，且与 batch 大小、序列长度几乎无关——这是推理成本的主要来源。

2.2 降低成本的两种思路

1. **减少权重**：量化（INT8/FP8/INT4）、模型蒸馏、剪枝。代价是精度损失。
2. **增加每步产出**：投机解码——每一步尝试产出多个 token，从而摊薄每次权重加载。

投机解码的独特之处在于：它**不改变输出分布**（数学上等价于原模型），因此是“免费”的加速（以额外草稿算力为代价）。这正是它区别于提前终止、近似采样等方案的根本属性。

3 投机解码原理

3.1 核心理念

投机解码把生成拆成三个阶段：

- 阶段 1：草稿（Draft）**：小模型自回归地逐个生成 k 个候选 token $D_1, \dots, D_k$ 。便宜，因为草稿小。
- 阶段 2：验证（Verify）**：大模型把 [前缀, $D_1, \dots, D_{k-1}$] 一次性批量前向，得到 k 个位置的输出分布。
- 阶段 3：接受（Accept）**：从首位置开始逐个判断草稿 token；只接受连续命中前缀，第一个不匹配处由大模型采样修正 token 补齐，丢弃后续草稿。

一次“引擎步”由此可产出 $1 + a \cdot k$ 个 token（ a 为接受率），而大模型权重仅加载 1 次（验证）+ 每步 1 次（常规解码的 base token）。

3.2 草稿阶段：逐个自回归生成

草稿 token 之间存在**依赖**： D_2 是在 D_1 已生成的前提下预测的，故草稿必须**严格自回归**、**逐个 token** 进行。其成本为 k 次小模型前向，但草稿模型远小于目标，总成本很低。

Listing 1: 本项目 DraftEngine::draft 核心循环

```

1 for (int i = 0; i < k; i++) {
2     Tensor h_out = run_layers(embed(input_token), 0, M, L + i, ...);
3     Tensor lg = lm_head_logits(rms_norm(h_out, norm, eps), embed);
4     int pred;
5     if (sample) { // 从 p_d 采样（概率接受必需）
6         softmax_inplace(pd, vocab);
7         pred = multinomial_draw(pd, vocab, rng);
8     } else {
9         pred = argmax(lg); // 贪心（确定性 argmax 接受）
10    }
11    out.push_back(pred);
12    input_token = pred; // 自回归依赖
13 }
```

3.3 验证阶段：一次批量前向

验证的关键在于大模型**无需逐个跑**。将 k 个候选拼成 $[\text{base}, D_1, \dots, D_{k-1}]$ 一次批量前向，第 i 行的注意力可看到前缀及前 $i-1$ 个候选，故一次前向同时得到 k 个位置的分布。在带宽受限硬件上， k 个 token 的批量前向与 1 个 token 的权重加载量相同——一次权重加载处理 k 个 token，这是加速的根源。

Listing 2: 本项目 forward_batched（批量验证原语）

```

1 // 行 i 位于绝对位置 L+i, attend 0..L+i-1（解码约定），预测 L+i+1
2 Tensor h_out = forward_batched(embed(verify_input),
3                                state.block_tables, L, /*write_kv=*/true);
```

验证同时以**权威方式**把 k 个位置的 K/V 写入缓存（复用与常规解码同一套 batched kernel），保证后续解码一致。

3.4 接受判据

3.4.1 严格 argmax 接受

最严格也最简单：仅当目标模型的 argmax 等于草稿 token 时接受：

$$\text{accept} \iff \arg \max_t p_t(t) = D_j \quad (1)$$

优点是输出与大模型**贪心解码逐 token 相同**（确定、易测）；缺点是跨规模双模型（如 0.5B 草稿 $\rightarrow$ 1.5B 目标）下两模型 argmax 几乎不重合，接受率塌缩到接近 0%。

3.4.2 概率接受（Leviathan）

Leviathan 的正确做法是**分布级接受**：以概率

$$p_{\text{accept}}(D_j) = \min\left(1, \frac{p_t(D_j)}{p_d(D_j)}\right) \quad (2)$$

接受草稿 token。当目标比草稿更看好该 token ($p_t \geq p_d$) 时必然接受；反之以该比值概率接受。期望接受率为：

$$a = \sum_t p_d(t) \cdot \min\left(1, \frac{p_t(t)}{p_d(t)}\right) = 1 - \text{TV}(p_d, p_t) \quad (3)$$

其中 $\text{TV}(p_d, p_t) = \frac{1}{2} \sum_t |p_d(t) - p_t(t)|$ 为总变差距离。接受率等于两模型分布的重叠程度，而非 argmax 是否重合——这是概率接受优于严格 argmax 的本质。

3.4.3 草稿必须采样而非贪心

概率接受要求草稿从 p_d 采样（而非贪心）。若草稿贪心，则永远提出其 argmax t^* ，接受概率退化为 $p_t(t^*)$ ；跨规模模型下 $p_t(t^*)$ 可能只有 10^{-5} 量级，接受率趋近 0。采样则让草稿有机会提出目标同样看好的 token ($p_t \geq p_d$ 者被无条件接受)。本项目实测：0.5B→1.5B 下，贪心草稿接受率 0%，采样草稿 96%。

Listing 3: 本项目概率接受实现

```
1 float p_dj = draft_probs[pd_off + drafts[j]];
2 float p_tj = p_t[drafts[j]];
3 float p_acc = (p_dj > 0) ? min(1.0f, p_tj / p_dj) : 0.0f;
4 accept_this = (p_acc >= 1.0f) || (u(rng) < p_acc);
```

3.5 残差修正与输出分布守恒定理

当草稿 token 被拒绝时，修正 token 必须从残差分布采样：

$$q(t) \propto \max(0, p_t(t) - p_d(t)) \quad (4)$$

这一选择并非随意，而是为了保证如下定理成立。

定理（输出分布守恒, Leviathan et al. 2023; Chen et al. 2023）：若草稿从 p_d 采样、接受概率为 $\min(1, p_t/p_d)$ 、拒绝后从残差 $\max(0, p_t - p_d)$ 归一化采样，则投机解码的输出分布恰等于目标分布 p_t 。

证明：设草稿提出 t 。

- 草稿提出 t 且被接受的概率： $p_d(t) \cdot \min(1, \frac{p_t(t)}{p_d(t)}) = \min(p_d(t), p_t(t))$ 。
- 拒绝概率： $1 - \sum_t \min(p_d(t), p_t(t))$ 。
- 拒绝后经残差修正发射 t 的概率： $\frac{\max(0, p_t(t) - p_d(t))}{1 - \sum_u \min(p_d(u), p_t(u))} \cdot (1 - \sum_u \min(p_d(u), p_t(u))) = \max(0, p_t(t) - p_d(t))$ 。
- 总发射概率： $\min(p_d, p_t) + \max(0, p_t - p_d) = p_t(t)$ 。 ■

工程意义：投机解码不是近似采样，而是数学上等价于原模型的采样器；严格 argmax 接受则是该定理在“草稿为点质量” ($p_d(t^*) = 1$) 时的退化情形。

Listing 4: 本项目残差修正实现

```
1 for (int i = 0; i < vocab; i++) {
2     float q = p_t[i] - p_d_full[pd_off + i];
```

```

3 logits_cpu[i] = (q > 0.0f) ? q : 0.0f; // max(0, p_t - p_d)
4 }
5 // 归一化后 multinomial_draw → correction token

```

3.6 连续前缀约束

草稿 token 相互依赖： D_{j+1} 的前提是 D_j 被接受。一旦在位置 j 拒绝， $D_{j+1} \dots D_k$ 的前提即失效，只能接受连续前缀 $D_1 \dots D_{j-1}$ ，并在 j 处采样修正 token。被丢弃后缀的 K/V 虽被写入，但注意力受序列长度限制不会读取，后续解码自然覆盖。

3.7 加速比的理论模型

设草稿与目标模型规模之比为 $r = M_{\text{draft}}/M_{\text{target}}$ ，接受率为 a ，草稿数为 k 。一次投机迭代：

- 目标侧：base 解码 1 次 + 批量验证 1 次，约 2 次权重加载（若验证取代解码则为 1 次）；
- 草稿侧： k 次草稿前向，约 $k \cdot r$ 次（相对目标权重）加载。

每迭代产出 $1 + a \cdot k$ 个 token，故相对纯解码（每 token 1 次加载）的加速比为：

$$\text{speedup} \approx \frac{1 + a \cdot k}{1 + k \cdot r + c} \quad (5)$$

其中 c 为验证/同步/CPU 开销。可见三个关键杠杆：**提高接受率 a** （草稿质量、采样）、**降低草稿比 r** （草稿更小）、**摊薄 c** （批量验证、验证即解码）。本项目实测： $a = 0.96$ 、 $k = 3$ 、 $r \approx 1/3$ ，理论加速约 $1.6\times$ ，实测 $0.98\times$ ——差距来自 c （双计算与 CPU 侧软 max），详见后续章节。

4 投机解码的发展历程

投机解码并非一蹴而就，其思想经历了从”块并行预测”到”草稿-验证范式”、再到”免草稿模型”的演进。

4.1 早期并行解码尝试（2018–2021）

自回归的串行性很早就被意识到是瓶颈。Stern et al. (2018) 提出**块并行解码（Blockwise Parallel Decoding）**：在解码器顶部加多个独立的”预测头”，一次性预测一整块 token，再贪心验证。但由于中间 token 的质量低、无理论保证，实际加速有限，未成为主流。

4.2 投机解码的独立提出（2022–2023）

2022–2023 年，三个研究组几乎同时独立提出”小模型草稿 + 大模型验证”的范式：

- **SpecInfer** (Miao et al., 2023)：以**树状草稿**（tree attention）支持多个草稿模型、多条候选路径一次验证，聚焦 serving 场景。
- **Leviathan et al.** (Google, 2023) 《Fast Inference from Transformers via Speculative Decoding》：给出**概率接受 + 残差修正**的完整理论，证明输出分布守恒。
- **Chen et al.** (DeepMind, 2023) 《Accelerating Large Language Model Decoding with Speculative Sampling》：独立得到同一结论。

两篇理论工作（Leviathan 与 Chen）分别证明了相同的核心定理（见 §3.5），确立了投机解码的数学基础。

4.3 工程化落地（2023–2024）

理论成熟后，主流推理引擎迅速集成：

- **vLLM**: SpecDecodeWorker，支持自投机与双模型，按草稿/验证/拒绝分阶段调度。
- **SGLang**: SpecDecodeDraftWorker，与 RadixAttention 前缀缓存结合。
- **TensorRT-LLM / NVIDIA**: 原生支持投机解码与草稿模型。
- **LightLLM（本项目）**: 自研 C++/CUDA 实现，聚焦 6GB 消费级 GPU 的显存与数值问题。

工程化的关键难题（也是本项目的重点）包括：KV cache 一致性、草稿与验证的 kernel 对齐、显存预算、结构化输出兼容。

4.4 后续演进（2024 至今）

投机解码仍在快速发展，主要方向：

- **EAGLE / EAGLE-2**: 草稿不再用单独的较小 transformer，而是在**特征层**做自回归 + 轻量预测头，接受率显著更高。
- **Medusa**: 在目标模型上直接挂多个解码头（multi-head），**免草稿模型**，约 2.2× 加速。
- **树注意力 / 多候选**: SpecInfer 的树、*Parallel Decoding via Hidden Transfer* 等，一次验证多条草稿路径。
- **静态草稿**: Google 的 *Block Verification Accelerates LLM Inference* 用预先选定的静态 token 块做草稿，连小模型都省去。

4.5 与本研究的关系

LightLLM 的定位是**验证经典范式（草稿-验证-接受）的完整工程实现**。我们刻意先实现标准的双模型/自投机路径，验证其理论承诺（输出守恒、接受率公式）在实际硬件上的成立程度；实测接受率 96% 与理论 $1 - TV$ 高度吻合。EAGLE/Medusa 等演进方向作为未来工作的参考（见 §9）。

5 系统架构设计

5.1 分层结构

LightLLM 将投机逻辑从核心解码路径解耦：

BatchMainLoop::step()

```
|-- engine_.step(batch, states)      // 干净解码契约（不感知投机）
|-- engine_.speculate(batch, states) // 投机编排层
    |-- draft_engine_->sync(state)  // 草稿自有 KV 同步到 seq_len
    |-- draft_engine_->draft(state, k, sample, &p_d)
    |-- forward_batched(...)        // 一次批量验证，写权威 KV
    |-- accept 循环 + 残差修正 + grammar 校验
```

收益：**step()** 保持投机无关，投机作为可插拔层，便于切换策略与单独测试。

5.2 DraftEngine 抽象

统一为 DraftEngine 接口，两种实现：

- **SelfSpecDraftEngine** (自投机)：目标前 M 层作草稿，共享权重但持自有 **KV cache**（不写目标权威缓存）；历史 KV 从目标缓存拷贝（而非重算），保证草稿上下文与目标逐位一致。
- **DualModelDraftEngine** (双模型)：独立小模型，自有权重与 KV。

Listing 5: DraftEngine 接口

```

1 class DraftEngine {
2     virtual void sync(RequestState& state) = 0; // 草稿 KV -> seq_len
3     virtual std::vector<int> draft(RequestState& state, int k,
4                                     bool sample,
5                                     std::vector<float>* p_d) = 0;
6     virtual void rollback(RequestState& state) = 0;
7     virtual std::vector<BlockAllocator*>& allocators() = 0;
8 };

```

5.3 批量验证原语 forward_batched 与 run_layers

验证必须复用与常规解码同一套 **batched kernel**，保证写入的 K/V 权威。**run_layers** 是核心原语：支持任意层区间与任意 allocator 集合，草稿引擎传入自有 allocator 即可复用同一前向逻辑。

KV 写入顺序：同一序列多行候选必须先写 K/V 再注意力（后一行可见前一行 KV），与单 token 解码”写后不读自身”一致。

5.4 KV cache 一致性

早期自投机直接用单 token 路径**重写目标权威缓存**，与 batched 解码路径数值不一致，导致投机”验证”的 token 非真贪心 token、输出整体偏离。修复：

1. 草稿永不写目标缓存，改用**自有 KV**；
2. 自投机草稿历史 KV 从目标缓存逐 **block** 拷贝；
3. 验证用 batched 前向一次性写权威 KV。

核心约束：目标缓存只允许 batched 路径写入。

5.5 结构化输出（Grammar）集成

投机 + 结构化输出需保证**每个发射 token 符合语法**：草稿 token 验证时逐 token **AcceptToken** 校验，非法即拒；修正 token 在语法掩码内采样；已接受 token 推进语法状态机。

5.6 采样参数化

temperature 改为按请求参数，语法只约束合法 token 集合（掩码），温度独立控制掩码内采样——对齐 vLLM/SGLang 的 constrained sampling。

6 显存管理

6GB 消费级 GPU 需同时驻留目标与草稿两套权重和 KV。KV 池预算支持三种模式：

模式	预算来源	适用场景
<code>kv_cache_mb > 0</code>	显式预算（每层）	用户精确控制
<code>budget_from_free</code>	构造时实际剩余显存	与目标共驻留的草稿
默认	总显存 - 本引擎权重 - 安全边距	独立引擎

关键设计：默认模式不用 `cudaMemGetInfo` 的 `free`（会被 CUDA 缓存分配器低估，顺序创建多引擎时误收缩），而用**总显存 - 权重字节**；草稿引擎用 `budget_from_free`（目标已驻留，`free` 反映真实余量），实现”权重后剩余空间分给两份 KV”。

实测 6GB 卡上：1.5B (fp16 $\approx$ 3.1GB) + 0.5B ($\approx$ 1GB) 共驻留，KV 自动收敛（目标 500MB、草稿 $\sim$ 280MB），不再 OOM。

7 实验与结果

实验环境：RTX 2060（6GB, sm_75），Qwen2.5-0.5B / Qwen2.5-1.5B。

7.1 自投机正确性（M = 24）

自投机以全模型为草稿（正确性验证配置）：

指标	常规解码	投机（M=24）
输出 token	10	10
步数	10	6
接受率	—	93.8%
输出一致性	—	逐 token 相同

输出一致性是 KV 一致性的硬门禁（`t1_tokens_identical`）。

7.2 0.5B 到 1.5B 双模型：接受判据的影响

接受判据	草稿行为	接受率	步数	加速比
严格 argmax	贪心	0%	—	0.36x
概率接受 + 顺序验证	采样	96%	—	0.79x
概率接受 + 批量验证	采样	96%	32→9	0.98x
概率接受 + 批量验证 + verify-as-decode	采样	94%	32→12	1.04x

结论：接受率 0%→96% 源于两个决策叠加——草稿**采样**与**概率接受**。批量验证使加速比 0.79x→0.98x（接近持平）。**verify-as-decode**（见 §8.4）把每步目标前向从 2 次降到 1 次，进一步将加速比突破到 **1.04x**（>1 即真实加速）。接受率 96% 与理论 $1 - \text{TV}(p_d, p_t)$ 吻合，验证了同族模型的分布接近性。

7.3 测试矩阵

测试	结果	覆盖
test_spec_decode	23/23	自投机正确性、接受率、语法
test_verify_incremental	16/16	双模型、内存拆分、概率接受
test_batch_loop	139/139	无投机回归
test_structured_e2e	12/12	常规语法路径
test_prefix_cache	176/176	前缀缓存

8 关键发现与设计决策

8.1 GEMM 批量尺寸敏感

本代码库的 GEMM/注意力 kernel 对批量尺寸数值敏感：单 token ($M=1$) 与批量 ($M=k$) 前向浮点结果有差异。同模型草稿在批量验证下接受率仅 6.2%，顺序验证下 93.8%。因此默认 `verify_batch=false`（顺序验证保正确性）；而概率接受对数值发散鲁棒（用 softmax 概率而非 argmax），可安全开启批量验证换取速度。

8.2 layer-slice 自投机在深层小模型上的局限

自投机以“前 M 层”为草稿时，接受率在 M 接近全模型处悬崖式跳变（0.5B 上 $M=18$ 仍 0%， $M=24$ 为 93.8%）。原因：中间层表示不直接预测下一个 token，深层模型的“最终决策”集中在后几层。因此 layer-slice 自投机更适合正确性验证而非实际加速；真实加速依赖独立小草稿模型或草稿头（EAGLE）。

8.3 草稿采样是接受率的分水岭

同族模型分布接近（TV 小），草稿采样后接受率 $= 1 - TV \approx 96\%$ ；贪心则困于自身 argmax，跨规模时接受率趋近 0%。这是本项目接受率从 0% 到 96% 的根本原因。

8.4 verify-as-decode：消除双计算（已实现）

早期流程中，每个投机步对 spec 请求做 2 次目标全模型前向：step() 解码 base token（位置 $L-1$ ）+ speculate() 批量验证草稿（位置 $L \dots L+k-1$ ）。这使得实测加速比（0.98x）低于理论模型（约 1.6x）。

verify-as-decode 让验证取代 step() 的常规解码：

- **batch loop 拆分**：prefill 条目 $\rightarrow$ step()，spec 解码条目 $\rightarrow$ speculate()（每步只做 1 次目标批量前向）。
- **位置语义**：base = 最后一个已提交 token（位置 $L = \text{seq_len}$ ），草稿从 base 预测 $D_1 \dots D_k$ ；验证输入 $[\text{base}@L, D_1, \dots, D_{k-1}]$ 从位置 L 起。草稿的 ‘pos = L+i’ 与验证结构保持不变（因为两种流程下 $L = \text{seq_len} = \text{base}$ 位置都成立）。
- **保留 pre-sample**：prefill 后处理仍预采样首 token（自包含注意力边界正确），spec 路径从第二个 token 开始，避免重推导首 token。
- **指标/终止**：metrics/TTFT/finish/release 从 decode 子批的状态更新；speculate() 补 ‘IsTerminated’（grammar 完成）与 ‘k<=0’（序列长度上限）兜底。

设计评审（智能体团队，2 视角）抓到一个关键正确性陷阱：若验证从位置 $L-1$ 起（含 D0 重推导），会覆盖 **prefill** 已提交的 **KV**——位置 $P-1$ 的 KV 由 prefill 以自包含注意力写入，而验证以自排除注意力重算，两者数值不同，且可能污染共享的前缀缓存块。采纳评审建议 **option(a)**：保留 pre-sample、验证从 base 位置起、绝不写位置 $L-1$ ，从而避免该问题。

实测：1.5B+0.5B 双模型（概率接受 + 批量验证）加速比从 **0.98x 突破到 1.04x**（接受率 94%，步数 32→12）——每步目标前向减半的直接收益。

代价：弱草稿（默认 $M=6$ 、接受率约 0%）下 verify-as-decode 去掉了 step() 每步的”免费 base”，退化为 1 token/步（更慢）——这是弱草稿的固有特性，非正确性 bug；真实收益在强接受率场景。

工程鲁棒性：为消除”KV 池耗尽导致崩溃”，`write_decode_kv` 增加块表越界检查（越界时优雅跳过并打印，而非野指针写崩溃）；测试在引擎创建前 `cudaDeviceReset()` 释放缓存内存。同模型双模型（0.5B+0.5B）在测试序列中因内存累积曾崩溃，修复后稳定通过；1.5B+0.5B 在 6GB 卡上内存临界，测试在可用内存不足时优雅跳过而非崩溃。

9 已知限制与未来工作

1. **verify-as-decode**（已实现，见 §8.4）：消除双计算，1.5B+0.5B 加速比 1.04x。剩余提升空间在于把每步目标前向进一步逼近理论下限。
2. **树注意力 / 多候选**：草稿并行生成多条候选序列（token 树），一次验证多条路径，进一步提高接受率与吞吐。
3. **免草稿模型方向**：参考 EAGLE（特征层自回归）与 Medusa（多头）思路，草稿可免去独立的较小 transformer。
4. **GPU 端残差**：当前草稿把完整 p_d ($k \times$ 词表) 传到 CPU 计算残差，是 CPU 侧热点；可改为 GPU 端残差 kernel。

10 结论

LightLLM 完整实现了投机解码的工程化闭环：小模型草稿逐个生成、大模型批量验证、连续前缀接受与残差修正，并通过 KV 一致性、显存预算、结构化输出等工程化处理使其在 6GB 消费级 GPU 上正确可用。实测跨规模双模型接受率从 0% 提升至 96%（与理论 $1 - TV$ 吻合），**verify-as-decode** 将加速比从 **0.98x 突破到 1.04x（真实加速）**，为树注意力、免草稿模型（EAGLE/Medusa）等后续优化奠定了正确的架构基础。本文档同步梳理了投机解码从块并行解码、独立理论提出、工程化落地到 EAGLE/Medusa 演进的全貌，明确了本项目在其中的定位与演进方向。

A 配置项

配置	默认	说明
<code>num_draft_tokens</code>	4	每步草稿 token 数 k
<code>draft_model_dir</code>	空	空 = 自投机；非空 = 双模型草稿
<code>draft_layers</code>	0	自投机草稿层数，0 = 自动 $n/4$
<code>verify_batch</code>	false	批量 ($M=k$) vs 顺序 ($M=1$) 验证
<code>probability_acceptance</code>	false	概率接受 vs 严格 argmax
<code>temperature</code>	1.0	按请求采样温度

B 测试清单

- `test_spec_decode` (23/23): 自投机正确性 (输出恒等)、接受率、速度机制、结构化输出首键合法; t1-t4 用 M=24 (高接受率) 验证 `verify-as-decode` 机制。
- `test_verify_incremental` (11/11): 双模型输出恒等、内存拆分、概率接受; 1.5B+0.5B 在内存足够时跑出 1.04x, 不足时优雅跳过。
- `test_batch_loop`(139/139)/`test_structured_e2e`(12/12)/`test_prefix_cache`(176/176): 无投机回归。